Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **1**

Subject: Lab on Java Programming Sign.of Teacher

Title: Implement a program that demonstrates program structure of java with use of arithmetical and 21 logical implementation.

Objective:

The program demonstrates basic **arithmetic operations** (addition, subtraction, multiplication, division) and **logical operations** (AND, OR, NOT).

Arithmetic Operations:

- The program demonstrates basic arithmetic operations such as addition, subtraction, multiplication, and division with integers.

Logical Operations:

- Logical operations such as AND (`&&`), OR (`||`), and NOT (`!`) are performed using boolean values (`true` and `false`).

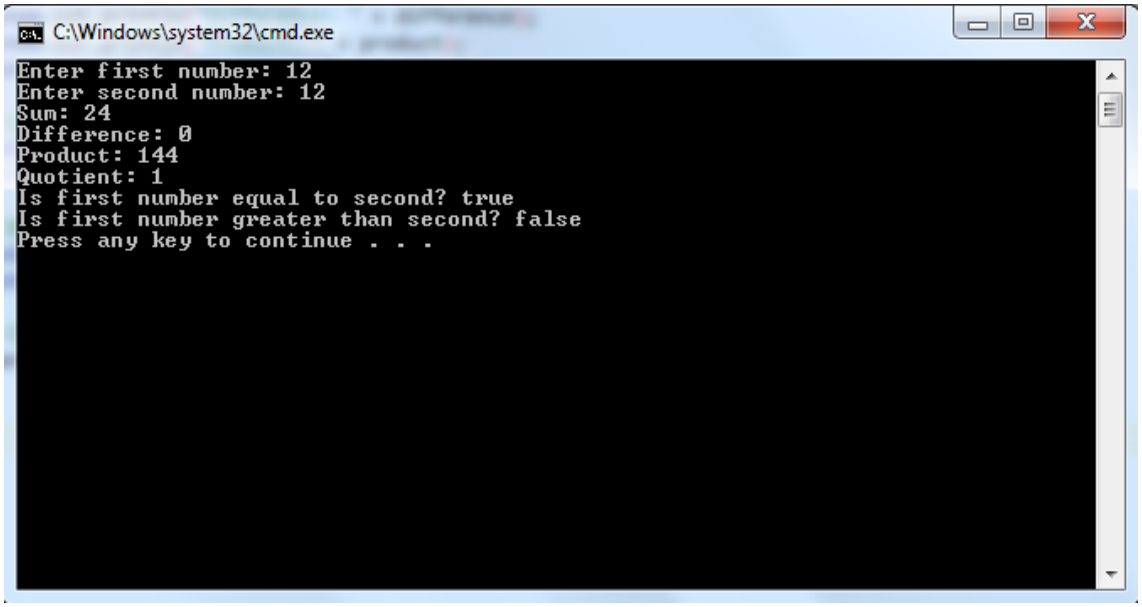
Steps for Java Program:

1. **Declare the Class**
   - Create a class that will hold the program.
2. **Declare the `main` Method**
   - Define the `main` method where execution starts.
3. **Import Required Packages (Optional)**
   - Import any necessary packages (e.g., `Scanner` for user input).
4. **Declare Variables**
   - Declare variables for arithmetic and logical operations (e.g., integers and booleans).
5. **Perform Arithmetic Operations**
   - Use arithmetic operators to perform operations like addition, subtraction, multiplication, division, and modulo.
6. **Perform Logical Operations**
   - Use logical operators (AND, OR, NOT) to perform logical comparisons.
7. **Display Results**
   - Output the results of both arithmetic and logical operations.
8. **Optional: Take User Input (if needed)**
   - Use `Scanner` to allow the user to input values for the operations.
9. **End the Program**
   - End the program after completing all operations and output.

1. **Implement a program that demonstrates program structure of java with use of arithmetical and 21 logical implementation.**

```
import java.util.Scanner;
public class SimpleArithmeticAndLogical {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);
      System.out.print("Enter first number: ");
      int num1 = scanner.nextInt();
      System.out.print("Enter second number: ");
      int num2 = scanner.nextInt();
      int sum = num1 + num2;
      int difference = num1 - num2;
      int product = num1 * num2;
      int quotient = (num2 != 0) ? num1 / num2 : 0;  // Prevent division by zero
      System.out.println("Sum: " + sum);
      System.out.println("Difference: " + difference);
      System.out.println("Product: " + product);
      System.out.println("Quotient: " + quotient);
          boolean isEqual = (num1 == num2);
      boolean isGreater = (num1 > num2);
      System.out.println("Is first number equal to second? " + isEqual);
      System.out.println("Is first number greater than second? " + isGreater);
      scanner.close();
   }
}
```

OUTPUT:



Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **2**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program that demonstrates string operations using String and String Buffer class.

Objective: To demonstrate string operations using the `String` and `StringBuffer` classes in Java, showing how to perform various string manipulations such as concatenation, comparison, and modification using both immutable (`String`) and mutable (`StringBuffer`) string objects.

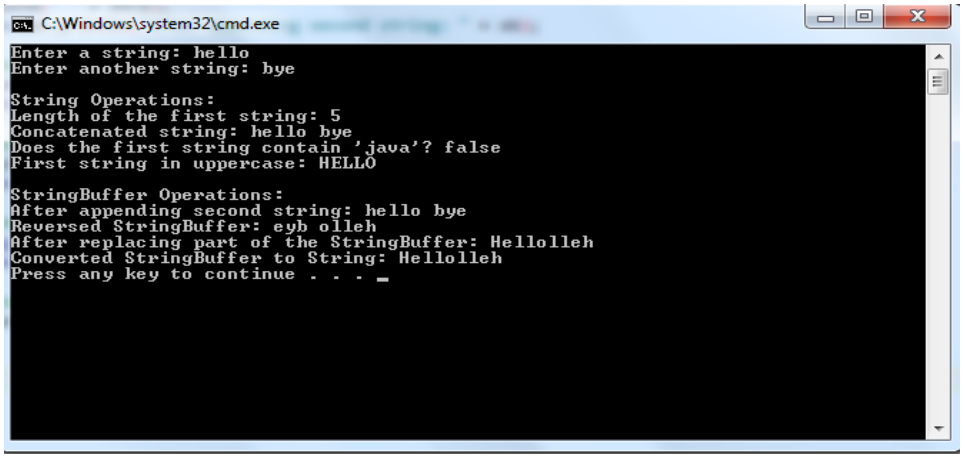
Steps for Java Program:

1. **Declare the Class**
   - Define a class, for example, StringBufferDemo, which will hold the string operations.
2. **Declare the main Method**
   - Define the main method as the entry point of the program.
3. **String Operations Using the String Class**
   - Create String objects for string manipulations.
   - Perform the following operations:
     - **Concatenation** using + or concat() method.
     - **Equality check** using equals() method.
     - **Case conversion** using toUpperCase() and toLowerCase().
     - **Length** using length().
     - **Substring** extraction using substring().
     - **Comparison** using compareTo().
4. **String Operations Using the StringBuffer Class**
   - Create StringBuffer objects for mutable string manipulations.
   - Perform the following operations:
     - **Append** using append() method.
     - **Insert** using insert() method.
     - **Reverse** using reverse() method.
     - **Replace** parts of the string using replace() method.
     - **Delete** characters using delete() or deleteCharAt().
5. **Display Results**
   - Use System.out.println() to display the results of all the string operations on the console.
6. **End the Program**
   - Close the main method and end the program execution.

Implement a program that demonstrates string operations using String and String Buffer class.

```
import java.util.Scanner;
public class StringBufferStringOperations {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);
      System.out.print("Enter a string: ");
      String str = scanner.nextLine();
      System.out.print("Enter another string: ");
      String str2 = scanner.nextLine();
      System.out.println("\nString Operations:");
      System.out.println("Length of the first string: " + str.length());
      String combined = str + " " + str2;
      System.out.println("Concatenated string: " + combined);
      boolean containsWord = str.contains("java");
      System.out.println("Does the first string contain 'java'? " + containsWord);
      String upperStr = str.toUpperCase();
      System.out.println("First string in uppercase: " + upperStr);
      StringBuffer sb = new StringBuffer(str);
      System.out.println("\nStringBuffer Operations:");
      sb.append(" " + str2);
      System.out.println("After appending second string: " + sb);
      sb.reverse();
      System.out.println("Reversed StringBuffer: " + sb);
      sb.replace(0, 5, "Hello");
      System.out.println("After replacing part of the StringBuffer: " + sb);
      String convertedStr = sb.toString();
      System.out.println("Converted StringBuffer to String: " + convertedStr);
      scanner.close();
   }
}
```

OUTPUT:



Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **3**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program that demonstrates inner class and static fields.

Objective: To demonstrate the use of **inner classes** and **static fields** in Java. This will show how an inner class can be defined inside an outer class and how static fields can be used to store class-level data.

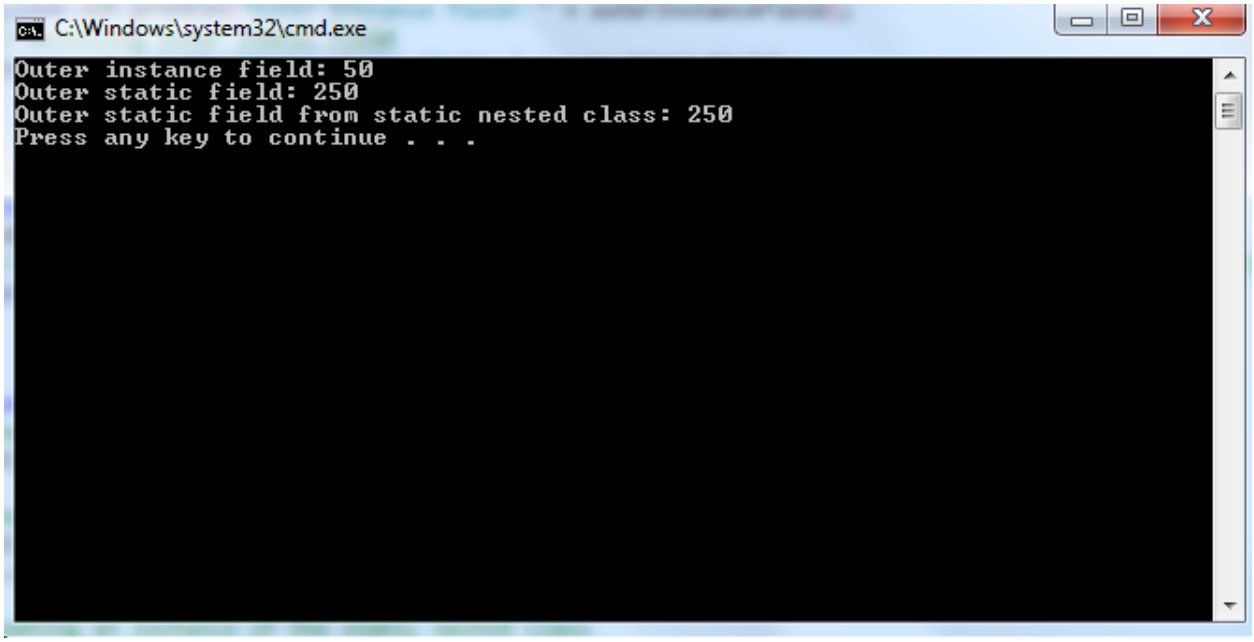
Steps for Java Program:

1. **Declare the Outer Class**
   - Create the outer class that will contain both the static field and the inner class.
2. **Declare the Static Field**
   - Inside the outer class, declare a static field (variable) that belongs to the class rather than an instance of the class.
3. **Declare the Inner Class**
   - Inside the outer class, define the inner class. The inner class can be non-static or static.
4. **Access Static Field from Inner Class**
   - Inside the inner class, demonstrate how to access the static field of the outer class.
5. **Create an Instance of the Outer Class in the main Method**
   - In the main method, create an instance of the outer class to demonstrate its functionality.
6. **Instantiate the Inner Class**
   - Instantiate the inner class either from an instance of the outer class (for non-static inner class) or directly using the outer class (for static inner class).
7. **Display Results**
   - Use System.out.println() to display the static field's value and any other outputs to the console.
8. **End the Program**
   - Complete the main method and close the program.

Implement a program that demonstrates inner class and static fields.

```
public class OuterClass {
    static int outerStaticField = 250;
    int outerInstanceField = 50;
    class InnerClass {
        void display() {
            System.out.println("Outer instance field: " + outerInstanceField);
            System.out.println("Outer static field: " + outerStaticField);
        }
    }
    static class StaticNestedClass {
        void display() {
            System.out.println("Outer static field from static nested class: " +
outerStaticField);
        }
    }
    public static void main(String[] args) {
            OuterClass outer = new OuterClass();
        OuterClass.InnerClass inner = outer.new InnerClass();
        inner.display(); // This will print fields from the outer class
        OuterClass.StaticNestedClass staticNested = new OuterClass.StaticNestedClass();
        staticNested.display(); // This will print the static field from the outer class
    }
}
```

OUTPUT:-



Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **4**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program that demonstrate inheritance, polymorphism

Objective: To demonstrate **inheritance** and **polymorphism** in Java. Inheritance allows one class to inherit fields and methods from another class, while polymorphism enables a method to behave differently based on the object it is acting upon.

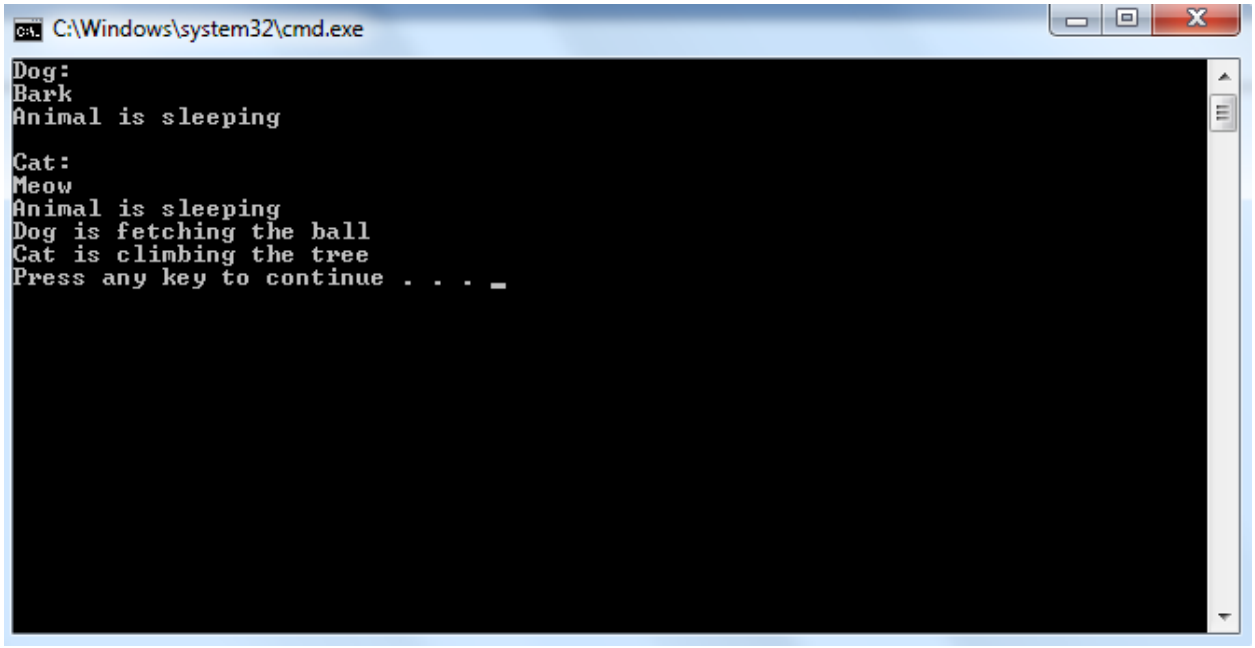
Steps for Java Program:

1. **Declare the Parent (Base) Class**
   - Define a parent class that will contain common fields and methods.
2. **Declare Inherited Methods**
   - In the parent class, declare methods that can be inherited by child classes.
3. **Declare the Child (Subclass) Class**
   - Define a child class that extends the parent class, inheriting its properties and behaviors.
4. **Override Methods in the Child Class (Polymorphism)**
   - In the child class, override the methods from the parent class to provide specific behavior (demonstrating polymorphism).
5. **Create Instances in the `main` Method**
   - In the `main` method, create objects of the parent class and the child class to demonstrate inheritance and polymorphism.
6. **Use Method Overriding to Demonstrate Polymorphism**
   - Call the overridden methods on instances of both parent and child classes and observe polymorphism (same method behaving differently depending on the object).
7. **Display Results**
   - Use `System.out.println()` to display the results of calling methods on both parent and child class objects.
8. **End the Program**
   - Complete the `main` method and close the program.

Implement a program that demonstrate inheritance, polymorphism

```
class Animal {
    public void sound() {
        System.out.println("Some animal sound");
    }
    public void sleep() {
        System.out.println("Animal is sleeping");
    }
}
class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("Bark");
    }
    public void fetch() {
        System.out.println("Dog is fetching the ball");
    }
}
class Cat extends Animal {
    @Override
    public void sound() {
        System.out.println("Meow");
    }
    public void climb() {
        System.out.println("Cat is climbing the tree");
    }
}
public class InheritancePolymorphismDemo {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        Animal myCat = new Cat();
        System.out.println("Dog:");
        myDog.sound();
        myDog.sleep();
        System.out.println("\nCat:");
        myCat.sound();
        myCat.sleep();
        if (myDog instanceof Dog) {
            Dog dog = (Dog) myDog;
            dog.fetch();
        }
        if (myCat instanceof Cat) {
            Cat cat = (Cat) myCat;
            cat.climb();
        }
    }
}
```

OUTPUT:-


C:\Windows\system32\cmd.exe
Dog:
Bark
Animal is sleeping
Cat:
Meow
Animal is sleeping
Dog is fetching the ball
Cat is climbing the tree
Press any key to continue . . .

Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **5**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program that demonstrates 2D shapes on frames.

Objective: To demonstrate the creation and display of **2D shapes** (such as rectangles, circles, and lines) on a **frame** in Java. This will involve using **Swing** for the graphical user interface (GUI) and **Graphics** to draw shapes.

Steps for Java Program:

1. **Import Required Packages**
   Import the necessary packages such as `javax.swing.*`, `java.awt.*`, and `java.awt.geom.*` to use Swing components and 2D graphics classes.
2. **Create a Class that Extends JPanel**
   Create a subclass of `JPanel` to perform custom drawing operations.
3. **Override paintComponent() Method**
   Inside the subclass, override the `paintComponent(Graphics g)` method to define the drawing logic.
4. **Convert Graphics to Graphics2D**
   Convert the `Graphics` object to `Graphics2D` for advanced 2D rendering.
5. **Set Colors and Strokes**
   Use methods like `setColor()` and `setStroke()` to define color and line thickness.
6. **Draw Various 2D Shapes**
   Use 2D shape classes such as:

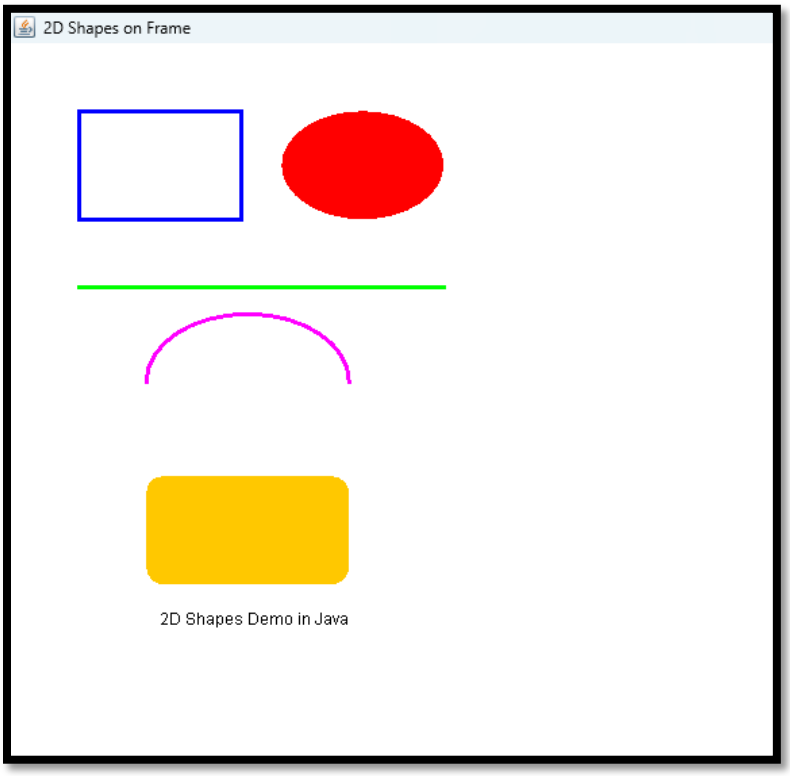
   - `Rectangle2D` for rectangles
   - `Ellipse2D` for ovals
   - `Arc2D` for arcs
   - `RoundRectangle2D` for rounded rectangles
   - `drawLine()` for straight lines

7. **Create a JFrame**
   In the main class, create an object of `JFrame` to display the panel.
8. **Add the JPanel to JFrame**
   Add the custom panel (where shapes are drawn) to the frame using `add()` method.
9. **Set Frame Properties**
   Set frame size, title, close operation, and make it visible using `setVisible(true)`.
10. **Execute the Program**
    Compile and run the program to display the frame with 2D shapes.

Implement a program that demonstrates 2D shapes on frames.

```
import javax.swing.*;
import java.awt.*;
import java.awt.geom.*;

class ShapePanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;

        // Set background color
        setBackground(Color.WHITE);

        // Set stroke (thickness)
        g2.setStroke(new BasicStroke(3));

        // Draw Rectangle
        g2.setColor(Color.BLUE);
        g2.draw(new Rectangle2D.Double(50, 50, 120, 80));

        // Draw Oval
        g2.setColor(Color.RED);
        g2.fill(new Ellipse2D.Double(200, 50, 120, 80));

        // Draw Line
        g2.setColor(Color.GREEN);
        g2.drawLine(50, 180, 320, 180);

        // Draw Arc
        g2.setColor(Color.MAGENTA);
        g2.draw(new Arc2D.Double(100, 200, 150, 100, 0, 180, Arc2D.OPEN));

        // Draw Rounded Rectangle
        g2.setColor(Color.ORANGE);
        g2.fill(new RoundRectangle2D.Double(100, 320, 150, 80, 25, 25));

        // Draw Text
        g2.setColor(Color.BLACK);
        g2.drawString("2D Shapes Demo in Java", 110, 430);
    }
}

public class ShapeDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("2D Shapes on Frame");
        frame.add(new ShapePanel());
        frame.setSize(400, 500);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

OUTPUT:



Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **6**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program that demonstrates color and fonts.

Objective: To demonstrate how to use **colors** and **fonts** in Java when drawing text and shapes on a **frame**. This will involve using **Swing** and **Graphics** for customizing the color and font styles in a graphical user interface.

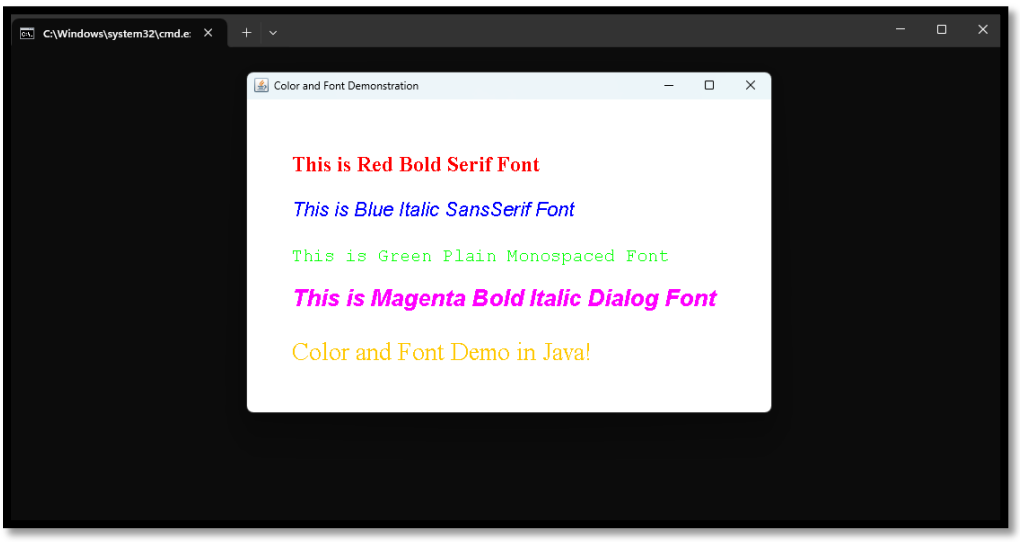
Steps for Java Program:

1. **Import Required Packages**
   Import the packages `javax.swing.*` and `java.awt.*` to use Swing components and graphics classes.
2. **Create a Class that Extends JPanel**
   Create a subclass of `JPanel` where the drawing and text rendering will be done.
3. **Override paintComponent() Method**
   Override the `paintComponent(Graphics g)` method to perform custom drawing operations.
4. **Set Background Color**
   Use `setBackground(Color.colorname)` method to set the background color of the panel.
5. **Set Text Color**
   Use `g.setColor(Color.colorname)` to change the color of the text before drawing it.
6. **Set Font Style and Size**
   Use the `Font` class constructor, for example:
   `g.setFont(new Font("Serif", Font.BOLD, 24));`
   to set the font family, style, and size.
7. **Display Text Using drawString()**
   Use `g.drawString("Text Message", x, y);` to display text at specific positions on the panel.
8. **Create a JFrame**
   In the main class, create a `JFrame` object to hold the panel.
9. **Add Panel to Frame**
   Add the custom panel (which draws text in various colors and fonts) to the frame using `add()` method.
10. **Set Frame Properties and Run**
    Set frame title, size, close operation, and make it visible using `setVisible(true)` to show the output window.

Implement a program that demonstrates color and fonts.

```
import javax.swing.*;
import java.awt.*;

public class ColorFontDemo extends JPanel {

   @Override
   protected void paintComponent(Graphics g) {
      super.paintComponent(g);

      // Set background color
      setBackground(Color.WHITE);

      // Draw text in different colors and fonts
      g.setColor(Color.RED);
      g.setFont(new Font("Serif", Font.BOLD, 24));
      g.drawString("This is Red Bold Serif Font", 50, 80);

      g.setColor(Color.BLUE);
      g.setFont(new Font("SansSerif", Font.ITALIC, 22));
      g.drawString("This is Blue Italic SansSerif Font", 50, 130);

      g.setColor(Color.GREEN);
      g.setFont(new Font("Monospaced", Font.PLAIN, 20));
      g.drawString("This is Green Plain Monospaced Font", 50, 180);

      g.setColor(Color.MAGENTA);
      g.setFont(new Font("Dialog", Font.BOLD + Font.ITALIC, 26));
      g.drawString("This is Magenta Bold Italic Dialog Font", 50, 230);

      g.setColor(Color.ORANGE);
      g.setFont(new Font("Serif", Font.PLAIN, 28));
      g.drawString("Color and Font Demo in Java!", 50, 290);
   }

   public static void main(String[] args) {
      JFrame frame = new JFrame("Color and Font Demonstration");
      ColorFontDemo panel = new ColorFontDemo();

      frame.add(panel);
      frame.setSize(600, 400);
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      frame.setVisible(true);
   }
}
```

OUTPUT:


C:\Windows\system32\cmd.e
Color and Font Demonstration
This is Red Bold Serif Font
This is Blue Italic SansSerif Font
This is Green Plain Monospaced Font
This is Magenta Bold Italic Dialog Font
Color and Font Demo in Java!

Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **7**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program to illustrate use of various swing components.

Objective: To demonstrate the use of various **Swing components** in Java, including buttons, labels, text fields, checkboxes, radio buttons, combo boxes, and panels to create a basic graphical user interface (GUI).

Steps for Java Program:

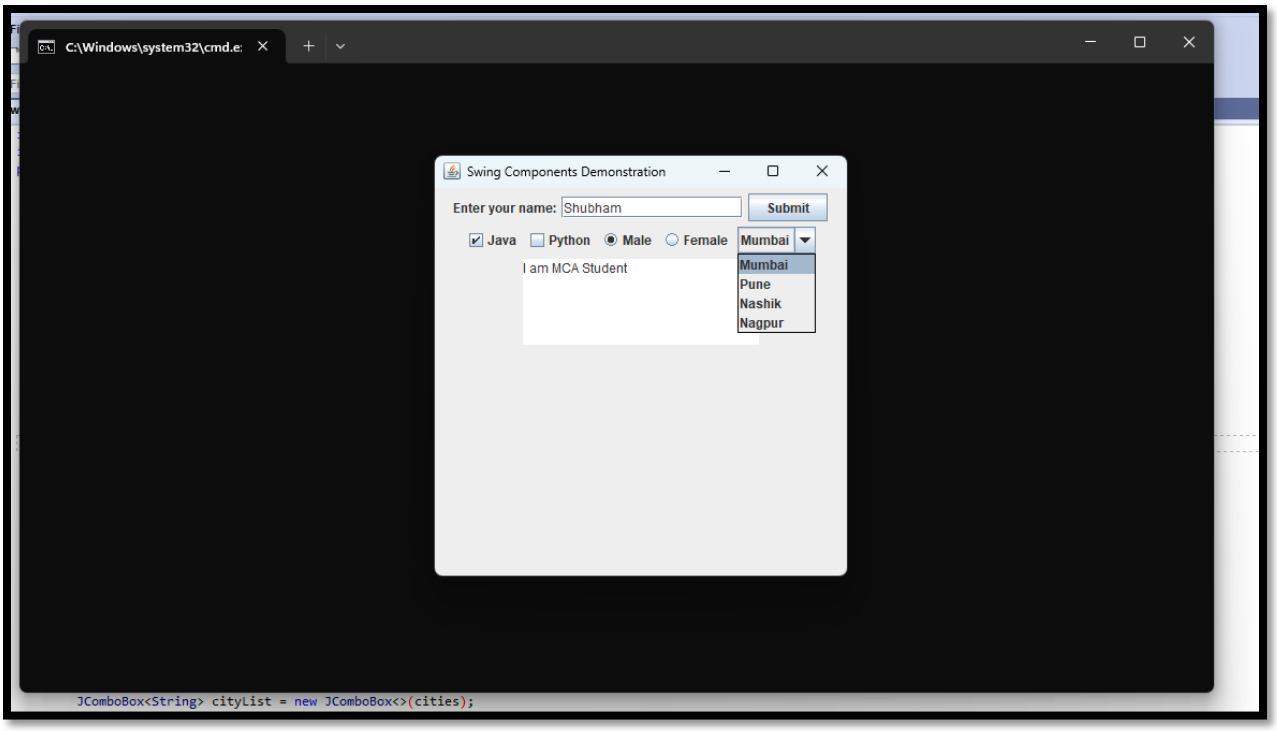
1. **Import Required Packages**
   Import the package `javax.swing.*` to use all the Swing classes and components.
2. **Create a Class that Extends JFrame**
   Create a class that extends `JFrame` to create the main application window.
3. **Create Swing Components**
   Declare and create objects of various Swing components such as:
   - `JLabel` – to display text or images
   - `JTextField` – to accept single-line text input
   - `JTextArea` – for multi-line text input
   - `JButton` – to perform an action
   - `JCheckBox` – to make multiple selections
   - `JRadioButton` – to select one option from many
   - `JComboBox` – to create a drop-down list
   - `JList` – to display a list of items
4. **Set Layout Manager**
   Use a layout manager such as `FlowLayout`, `BorderLayout`, or `GridLayout` to arrange components in the frame.
5. **Add Components to the Frame**
   Add all Swing components to the frame using the `add()` method.
6. **Add Event Handling (Optional)**
   Attach event listeners like `ActionListener` to components (such as buttons) to handle user actions.
7. **Set Frame Properties**
   - Set the title of the frame using `setTitle()`
   - Set the size using `setSize()`
   - Use `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` to close the window properly
   - Make the frame visible using `setVisible(true)`
8. **Compile and Run the Program**
   Compile the program and run it to see various Swing components displayed in the window.

Implement a program to illustrate use of various swing components.

```
import javax.swing.*;
import java.awt.*;
public class SwingComponentsDemo extends JFrame {
    public SwingComponentsDemo() {
        // Set title for the frame
        setTitle("Swing Components Demonstration");
        // Set layout manager
        setLayout(new FlowLayout());
        // 1. JLabel
        JLabel label = new JLabel("Enter your name:");
        add(label);
        // 2. JTextField
        JTextField textField = new JTextField(15);
        add(textField);
        // 3. JButton
        JButton button = new JButton("Submit");
        add(button);
        // 4. JCheckBox
        JCheckBox checkBox1 = new JCheckBox("Java");
        JCheckBox checkBox2 = new JCheckBox("Python");
        add(checkBox1);
        add(checkBox2);
        // 5. JRadioButton (Radio Buttons)
        JRadioButton male = new JRadioButton("Male");
        JRadioButton female = new JRadioButton("Female");
        // Group the radio buttons so only one can be selected
        ButtonGroup genderGroup = new ButtonGroup();
        genderGroup.add(male);
        genderGroup.add(female);
        add(male);
        add(female);
        // 6. JComboBox (Drop-down)
        String cities[] = {"Mumbai", "Pune", "Nashik", "Nagpur"};
        JComboBox<String> cityList = new JComboBox<>(cities);
        add(cityList);
        // 7. JTextArea
        JTextArea textArea = new JTextArea("Comments...", 5, 20);
        add(textArea);
        // Set frame size and default close operation
        setSize(400, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
```

```
        }
        // Main method
        public static void main(String[] args) {
            new SwingComponentsDemo();
        }

}
```

OUTPUT:



Godavari Institute Of Management & Research, Jalgaon

Name: __ _ _ _ _ _ _ Roll No: _ _ _ __

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **8**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program that demonstrates use of dialog box and menus.

Objective: To demonstrate the use of **dialog boxes** and **menus** in Java using **Swing**. Dialog boxes are used to display information or request input from the user, while menus provide an interactive interface for users to choose from various actions.

Steps for Java Program:

1. **Import Required Packages**
   Import the packages required for Swing components and event handling:
   `javax.swing.*` → for GUI components like JMenu, JMenuItem, JOptionPane, etc.
   `java.awt.event.*` → for handling menu click events.
2. **Create a Class that Extends JFrame**
   Create a class (for example, `MenuDialogDemo`) that extends **JFrame** to create the main application window.
   Also, implement **ActionListener** to handle user interactions.
3. **Create Menu Bar and Menus**
   Declare and create objects of Swing menu components:
   - **JMenuBar** – to hold menus
   - **JMenu** – to create menus like File and Help
   - **JMenuItem** – to create options like Open, Save, Exit, About
4. **Add Menu Items to Menus**
   Add all menu items to their respective menus using the `add()` method.
   For example:
5. **Add Menus to Menu Bar**
   Attach all menus to the **menu bar**.
6. **Set the Menu Bar to the Frame**
   Set the menu bar for the frame using:
7. **Add Event Handling**
   Attach **ActionListener** to each menu item to perform actions when clicked.
   `openItem.addActionListener(this);`
   Handle the events using the `actionPerformed()` method.
8. **Display Dialog Boxes**
   Use **JOptionPane** class to show dialog boxes like:
   These dialog boxes show messages or ask for user confirmation.
9. **Set Frame Properties**
   Set frame title, size, layout, and visibility:
10. **Compile and Run the Program**
    - Save the file as **MenuDialogDemo.java**
    - Compile the program using **Tools → Compile Java (Ctrl + 1)**
    - Run the program using **Tools → Run Java Application (Ctrl + 2)**

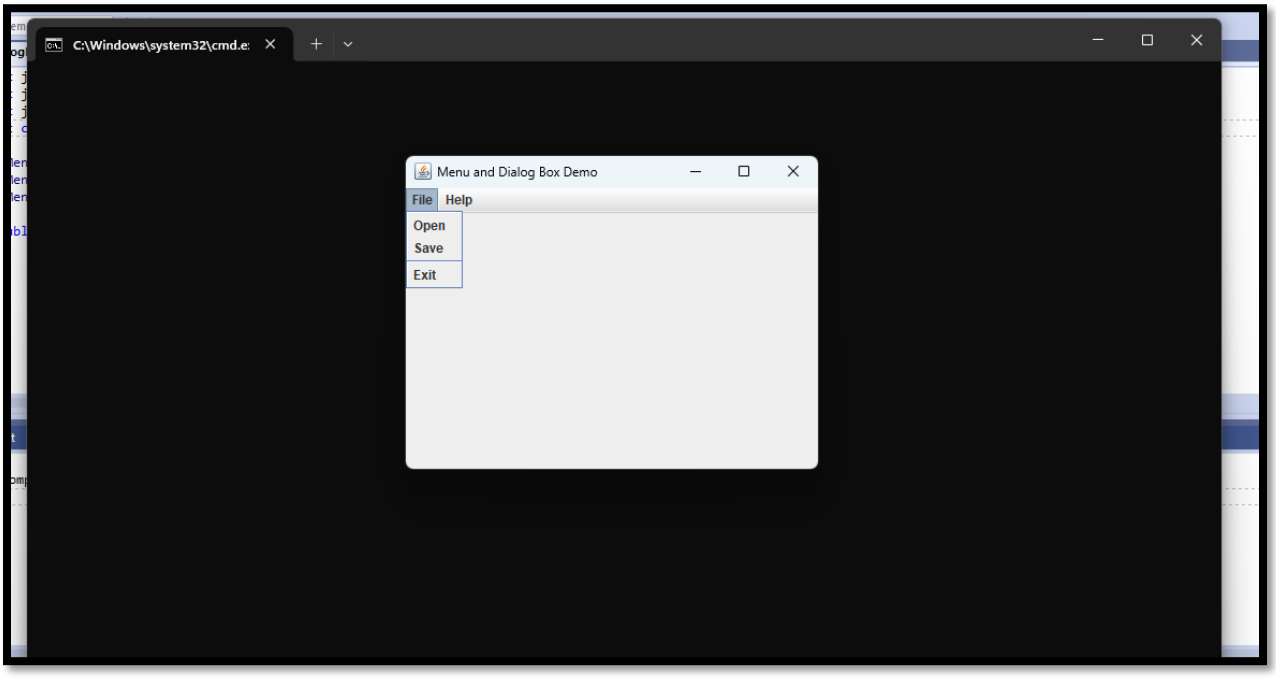
The frame will appear with menus and dialog boxes that open when menu items are selected.

Implement a program that demonstrates use of dialog box and menus.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class MenuDialogDemo extends JFrame implements ActionListener {
   JMenuBar menuBar;
   JMenu fileMenu, helpMenu;
   JMenuItem openItem, saveItem, exitItem, aboutItem;
   public MenuDialogDemo() {
      // Frame title
      setTitle("Menu and Dialog Box Demo");
      // Create Menu Bar
      menuBar = new JMenuBar();
      // Create Menus
      fileMenu = new JMenu("File");
      helpMenu = new JMenu("Help");
      // Create Menu Items
      openItem = new JMenuItem("Open");
      saveItem = new JMenuItem("Save");
      exitItem = new JMenuItem("Exit");
      aboutItem = new JMenuItem("About");
      // Add items to File menu
      fileMenu.add(openItem);
      fileMenu.add(saveItem);
      fileMenu.addSeparator(); // Line separator
      fileMenu.add(exitItem);
      // Add item to Help menu
      helpMenu.add(aboutItem);
      // Add Menus to MenuBar
      menuBar.add(fileMenu);
      menuBar.add(helpMenu);
      // Set the Menu Bar
      setJMenuBar(menuBar);
      // Add Action Listeners
      openItem.addActionListener(this);
      saveItem.addActionListener(this);
      exitItem.addActionListener(this);
      aboutItem.addActionListener(this);
      // Frame settings
      setSize(400, 300);
      setLayout(new FlowLayout());
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
        setVisible(true);
    }

    // Action Events for Menu Items
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == openItem) {
            JOptionPane.showMessageDialog(this, "Open option selected!");
        } else if (e.getSource() == saveItem) {
            JOptionPane.showMessageDialog(this, "File Saved Successfully!");
        } else if (e.getSource() == exitItem) {
            int confirm = JOptionPane.showConfirmDialog(this, "Are you sure you want to
exit?", "Exit Confirmation", JOptionPane.YES_NO_OPTION);
            if (confirm == JOptionPane.YES_OPTION) {
                System.exit(0);
            }
        } else if (e.getSource() == aboutItem) {
            JOptionPane.showMessageDialog(this, "This program demonstrates menus and
dialog boxes.\nCreated by: [Your Name]");
        }
    }

    // Main Method
    public static void main(String[] args) {
        new MenuDialogDemo();
    }
}
```

Output:



Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **9**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program that demonstrates event handling for various types of events.

Objective: To demonstrate **event handling** in Java for various types of events, such as action events, mouse events, key events, and window events, using **Swing** components. This will show how to capture and respond to user actions like button clicks, mouse movements, keyboard input, and window interactions.

Steps for Java Program:

1. **Import Required Packages**
   Import the packages required for GUI components and event handling: javax.swing.*, java.awt.*, and java.awt.event.*.
2. **Create a Class that Extends JFrame**
   Create a class that extends **JFrame** to create the main application window and implements listener interfaces like **ActionListener**, **MouseListener**, or **KeyListener**.
3. **Create GUI Components**
   Declare and create objects of various Swing components such as:
   - **JButton** – for button click events
   - **JTextField** – for key events
   - **JLabel** – to display messages or feedback
4. **Register Event Listeners**
   Attach the appropriate event listeners to the components to handle different types of events:
   - addActionListener() for button clicks
   - addMouseListener() for mouse events
   - addKeyListener() for keyboard events
5. **Define Event Handling Methods**
   Implement the methods of each listener interface such as:
   - actionPerformed(ActionEvent e)
   - mouseClicked(MouseEvent e)
   - keyPressed(KeyEvent e)

   Each method defines what action should be taken when the event occurs.
6. **Add Components to the Frame**
   Use the add() method to place all components like labels, buttons, and text fields onto the frame.
7. **Set Frame Properties**
   Set frame title, size, layout, close operation, and make it visible using:
   - setTitle()
   - setSize()
   - setDefaultCloseOperation()
   - setVisible(true)

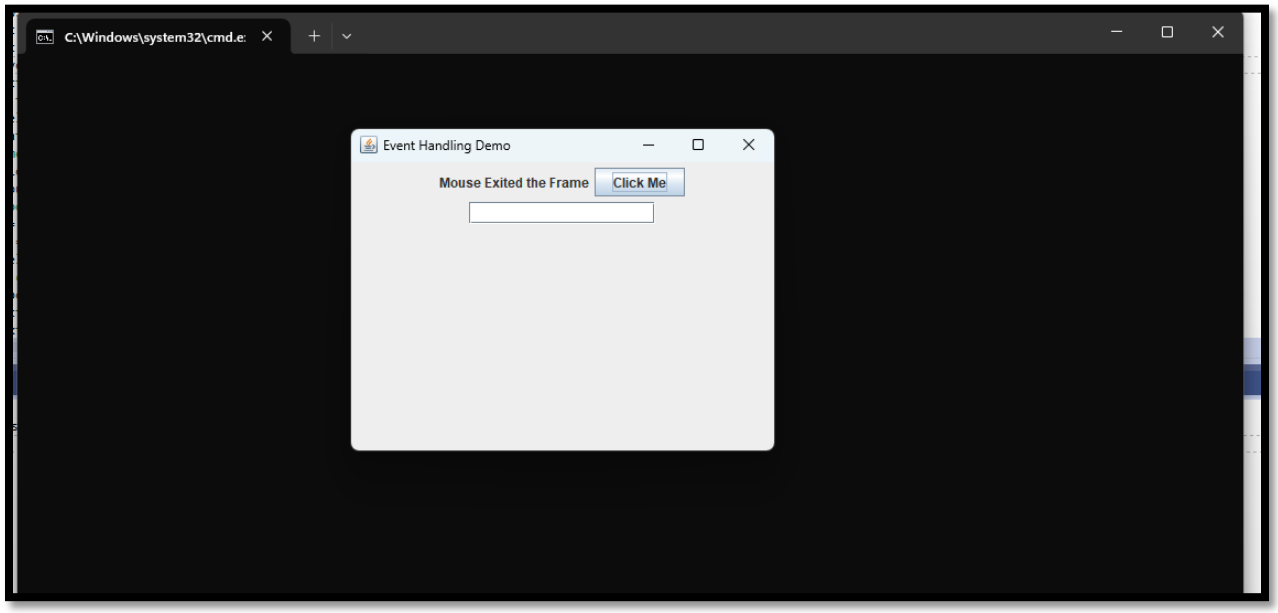
8. **Compile and Run the Program**
   Compile and run the program to observe how different events (button clicks, mouse movement, key presses) trigger actions in the application.

Implement a program that demonstrates event handling for various types of events.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class EventHandlingDemo extends JFrame implements ActionListener,
MouseListener, KeyListener {
   JButton button;
   JTextField textField;
   JLabel label;
   public EventHandlingDemo() {
      // Frame title
      setTitle("Event Handling Demo");
      setLayout(new FlowLayout());
      // Components
      label = new JLabel("Perform any action:");
      button = new JButton("Click Me");
      textField = new JTextField(15);
      // Add components
      add(label);
      add(button);
      add(textField);
      // Add Listeners
      button.addActionListener(this);   // For button click
      addMouseListener(this);           // For mouse events on frame
      textField.addKeyListener(this);   // For keyboard events
      // Frame settings
      setSize(400, 300);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      setVisible(true);
   }
   // Action Event
   public void actionPerformed(ActionEvent e) {
      label.setText("Button Clicked!");
   }
   // Mouse Events
   public void mouseClicked(MouseEvent e) {
      label.setText("Mouse Clicked at (" + e.getX() + "," + e.getY() + ")");
   }
   public void mousePressed(MouseEvent e) {}
```

```
    public void mouseReleased(MouseEvent e) {}
    public void mouseEntered(MouseEvent e) {
        label.setText("Mouse Entered the Frame");
    }
    public void mouseExited(MouseEvent e) {
        label.setText("Mouse Exited the Frame");
    }
    // Key Events
    public void keyPressed(KeyEvent e) {
        label.setText("Key Pressed: " + e.getKeyChar());
    }
    public void keyReleased(KeyEvent e) {}
    public void keyTyped(KeyEvent e) {}
    // Main Method
    public static void main(String[] args) {
        new EventHandlingDemo();
    }
}
```

Output:



Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **10**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program to illustrate multithreading.

Objective: To demonstrate the concept of multithreading in Java by creating multiple threads that execute concurrently to perform different tasks simultaneously.

Steps:

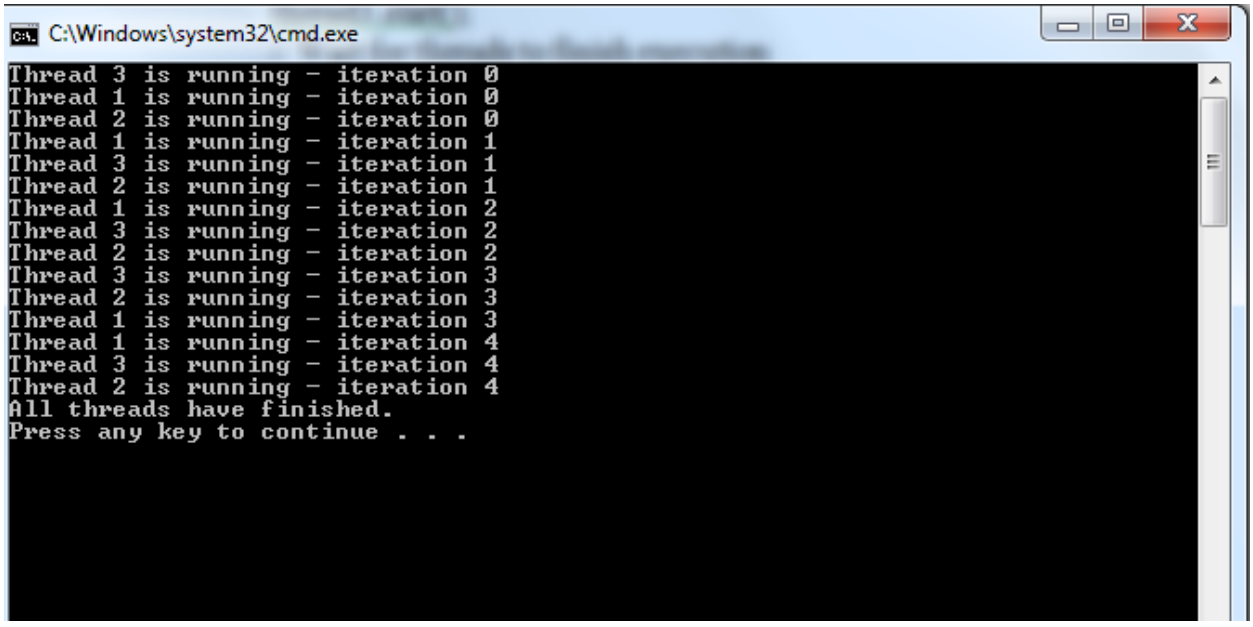
1. **Create a class that implements the `Runnable` interface**.
   - Define a `run()` method to specify the task that the thread will execute.
2. **Create a `Thread` object**.
   - Instantiate a `Thread` object and pass the `Runnable` implementation to it.
3. **Override the `run()` method** in the `Runnable` implementation.
   - Include the task logic you want each thread to perform.
4. **Start the threads** using the `start()` method.
   - This will invoke the `run()` method in each thread concurrently.
5. **Use `Thread.sleep()` (optional)** to simulate delays and demonstrate concurrency.
6. **(Optional) Synchronize threads** if accessing shared resources to avoid data inconsistency.
7. **Monitor the threads** by printing outputs in the `run()` method to observe parallel execution.

Implement a program to illustrate multithreading.

```
class MyThread extends Thread {
    private int threadNumber;
    public MyThread(int threadNumber) {
        this.threadNumber = threadNumber;
    }
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            try {
                // Simulate some work with sleep
                Thread.sleep(500);
                System.out.println("Thread " + threadNumber + " is running - iteration " +
i);
            } catch (InterruptedException e) {
                System.out.println("Thread " + threadNumber + " was interrupted.");
            }
        }
    }
```

```
}
public class MultithreadingExample {
    public static void main(String[] args) {
        // Create and start multiple threads
        MyThread thread1 = new MyThread(1);
        MyThread thread2 = new MyThread(2);
        MyThread thread3 = new MyThread(3);
        // Start the threads
        thread1.start();
        thread2.start();
        thread3.start();
        // Wait for threads to finish execution
        try {
            thread1.join();
            thread2.join();
            thread3.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread was interrupted.");
        }
        System.out.println("All threads have finished.");
    }
}
```

Output:



Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **11**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program to illustrate exception handling.

Objective: To demonstrate exception handling in Java by creating a program that handles runtime and compile-time exceptions, ensuring the program can continue running even when an error occurs.

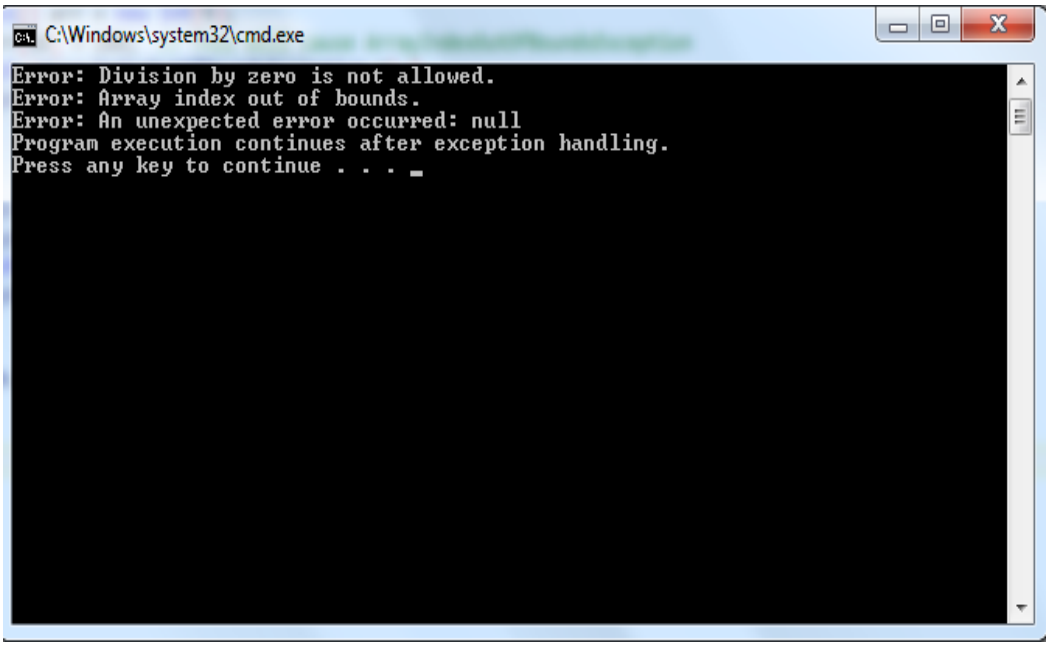
Steps:

1. **Identify the potential areas where exceptions might occur**.
   - Think of operations that can cause exceptions, such as file reading, database access, array indexing, division by zero, etc.
2. **Use `try-catch` block** to handle exceptions:
   - Wrap the code that might throw an exception in the `try` block.
   - Use `catch` blocks to handle specific exceptions.
3. **Handle different types of exceptions**:
   - Use multiple `catch` blocks for different types of exceptions (e.g., `ArithmeticException`, `ArrayIndexOutOfBoundsException`, etc.).
   - Optionally, use a generic `Exception` to catch any unforeseen exceptions.
4. **Use `finally` block** (optional but recommended):
   - The `finally` block will execute regardless of whether an exception was thrown or not (e.g., for cleanup operations like closing files or releasing resources).
5. **Throw exceptions explicitly** (optional):
   - Use `throw` to manually throw exceptions when necessary (e.g., custom exception handling).
6. **Test with various exceptions**:
   - Test scenarios where exceptions occur, such as invalid user input, dividing by zero, etc.
7. **Log or print error messages**:
   - Inside the `catch` block, log or print meaningful messages to help identify the problem.
8. **Program termination**:
   - Ensure the program can either recover or gracefully terminate when an exception is handled.

Implement a program to illustrate exception handling.

```
public class ExceptionHandlingExample {
    public static void main(String[] args) {

        try {
            int result = 10 / 0;  // This will cause ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero is not allowed.");
        }

        try {
            int[] arr = new int[5];
            arr[10] = 100;  // This will cause ArrayIndexOutOfBoundsException
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Array index out of bounds.");
        }

        try {
            String text = null;
            System.out.println(text.length());  // This will cause NullPointerException
        } catch (Exception e) {
            System.out.println("Error: An unexpected error occurred: " + e.getMessage());
        }

        System.out.println("Program execution continues after exception handling.");
    }
}
```

Output:



Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no:**12**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program to demonstrate use of File class.

Objective: To demonstrate the use of the `File` class in Java for performing basic file operations such as creating, reading, writing, and deleting files.

Steps:

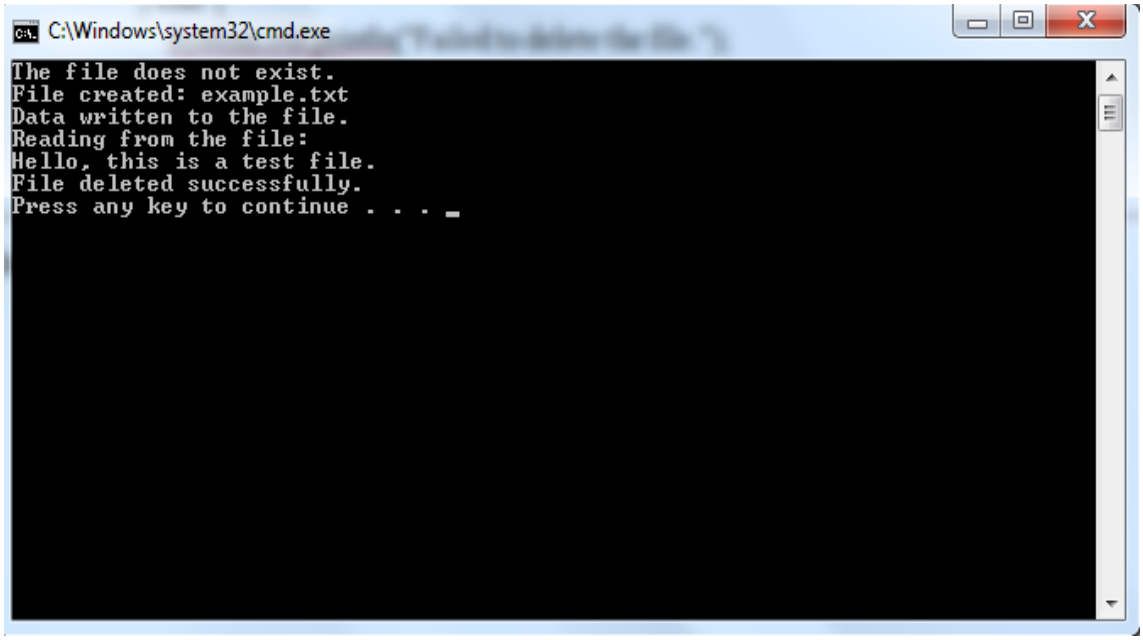
1. **Import the java.io.File class**.
   - Ensure you have the necessary import statement: import java.io.File;
2. **Create a File object**.
   - Instantiate a File object by providing the file path (relative or absolute) as a string.
3. **Check if the file exists**:
   - Use the exists() method to check if a file already exists at the specified location.
4. **Create a new file**:
   - Use the createNewFile() method to create a new file if it doesn't already exist.
5. **Check if the file is a directory or a file**:
   - Use isDirectory() and isFile() methods to check if the File object represents a directory or a regular file.
6. **Write data to a file**:
   - Use classes like FileWriter or BufferedWriter to write text data to the file.
7. **Read data from a file**:
   - Use FileReader or BufferedReader to read the content from the file.
8. **Delete a file**:
   - Use the delete() method to delete the file from the filesystem.
9. **Check file permissions**:
   - Use methods like canRead(), canWrite(), and canExecute() to check file access permissions.
10. **List files in a directory**:
    - Use the list() method to get the names of all files in a directory.
11. **Use length() to get file size**:
    - Call the length() method to get the size of the file in bytes.
12. **Ensure proper exception handling**:
    - Handle IOException using try-catch blocks to manage any IO-related issues during file operations.

Implement a program to demonstrate use of File class.

```
import java.io.File;
import java.io.FileWriter;
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;
public class FileClassExample {
   public static void main(String[] args) {
      // Specify the file path
      String filePath = "example.txt";
      // Create a File object
      File file = new File(filePath);
      // Example 1: Check if the file exists
      if (file.exists()) {
         System.out.println("The file already exists.");
      } else {
         System.out.println("The file does not exist.");
         try {
            // Example 2: Create the file
            if (file.createNewFile()) {
               System.out.println("File created: " + file.getName());
            } else {
               System.out.println("File already exists.");
            }
         } catch (IOException e) {
            System.out.println("An error occurred while creating the file.");
            e.printStackTrace();
         }
      }

      // Example 3: Write data to the file
      try (FileWriter writer = new FileWriter(file)) {
         writer.write("Hello, this is a test file.");
         System.out.println("Data written to the file.");
      } catch (IOException e) {
         System.out.println("An error occurred while writing to the file.");
         e.printStackTrace();
      }
      // Example 4: Read data from the file
      try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
         String line;
         System.out.println("Reading from the file:");
         while ((line = reader.readLine()) != null) {
            System.out.println(line);
         }
      } catch (IOException e) {
         System.out.println("An error occurred while reading the file.");
         e.printStackTrace();
```

```
        }

        // Example 5: Delete the file
        if (file.delete()) {
            System.out.println("File deleted successfully.");
        } else {
            System.out.println("Failed to delete the file.");
        }
    }
}
```

Output:



Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **13**

Subject: Lab on Java Programming Sign. of Teacher:

Title: Implement a program that demonstrates JDBC on application.

Objective: To demonstrate the use of JDBC (Java Database Connectivity) for connecting to a database, executing SQL queries, and performing basic database operations such as inserting, updating, and retrieving data in Java.

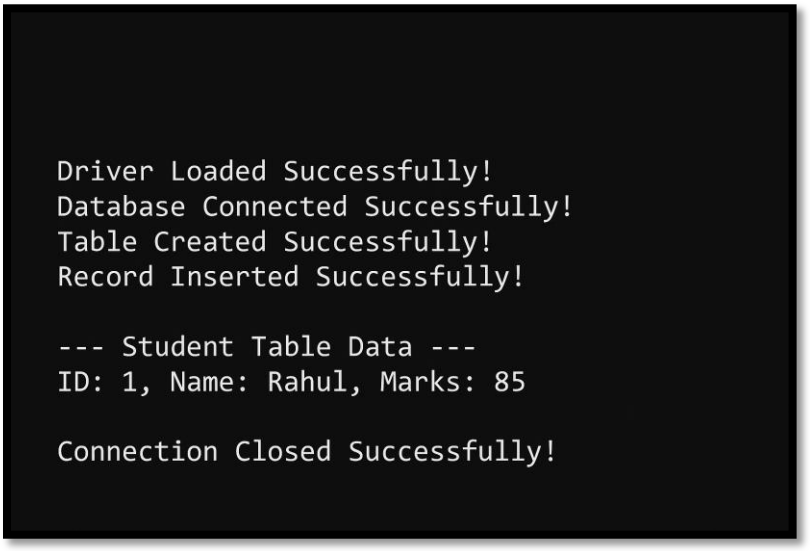
Steps to Demonstrate JDBC in Java:

1. **Import Required Packages**
   Import the java.sql.* package to use JDBC classes such as Connection, Statement, ResultSet, and DriverManager.
2. **Load the JDBC Driver**
   Load the JDBC driver class for the specific database (for example, MySQL) using the Class.forName() method.
   This step registers the driver with the DriverManager.
3. **Establish Database Connection**
   Use the DriverManager.getConnection() method to connect the Java application with the database by providing the database URL, username, and password.
4. **Create Statement Object**
   Create a Statement or PreparedStatement object to send SQL queries to the database.
5. **Execute SQL Queries**
   - Use the statement object to execute different SQL commands:
   - executeUpdate() → for **INSERT**, **UPDATE**, or **DELETE** queries.
   - executeQuery() → for **SELECT** queries.
6. **Process the Result**
   If a SELECT query is executed, process the result using a ResultSet object to retrieve and display the records from the database.
7. **Close the Connection**
   After all operations are completed, close the ResultSet, Statement, and Connection objects to free up database resources.
8. **Compile and Run the Program**
   Compile the Java program and run it.
   The application will connect to the database, execute the SQL statements, and display the results on the screen.

Implement a program that demonstrates JDBC on application.

```
import java.sql.*;  // Import JDBC package
public class JDBCDemo {
   public static void main(String[] args) {
      // Database credentials
      String url = "jdbc:mysql://localhost:3306/testdb"; // Database name = testdb
      String user = "root"; // Your MySQL username
      String password = ""; // Your MySQL password (keep blank if none)
      try {
         // 1. Load MySQL JDBC Driver
         Class.forName("com.mysql.cj.jdbc.Driver");
         System.out.println("Driver Loaded Successfully!");
         // 2. Establish Connection
         Connection con = DriverManager.getConnection(url, user, password);
         System.out.println("Database Connected Successfully!");
         // 3. Create Statement
         Statement stmt = con.createStatement();
         // 4. Create Table (if not exists)
         String createTable = "CREATE TABLE IF NOT EXISTS student (id INT PRIMARY
KEY, name VARCHAR(50), marks INT)";
         stmt.executeUpdate(createTable);
         System.out.println("Table Created Successfully!");
         // 5. Insert Data
         String insertQuery = "INSERT INTO student VALUES (1, 'Rahul', 85)";
         stmt.executeUpdate(insertQuery);
         System.out.println("Record Inserted Successfully!");
         // 6. Retrieve Data
         String selectQuery = "SELECT * FROM student";
         ResultSet rs = stmt.executeQuery(selectQuery);
         System.out.println("\n--- Student Table Data ---");
         while (rs.next()) {
            System.out.println("ID: " + rs.getInt("id") +
                               ", Name: " + rs.getString("name") +
                               ", Marks: " + rs.getInt("marks"));
         }
         // 7. Close Connection
         con.close();
         System.out.println("\nConnection Closed Successfully!");
      } catch (Exception e) {
         System.out.println("Error: " + e.getMessage());
      }
   }
}
```

OUTPUT:-


Driver Loaded Successfully!
Database Connected Successfully!
Table Created Successfully!
Record Inserted Successfully!
--- Student Table Data ---
ID: 1, Name: Rahul, Marks: 85
Connection Closed Successfully!

Godavari Institute Of Management & Research, Jalgaon

Name: _ Roll No: _ _ _ _ _

Date of Performance: _ _ /_ _/20_ _ Batch: _ _ _ _ _ _

Class: M.C.A. (I) Practical no: **14**

Subject: Lab on Java Programming Sign. of Teacher**:**

Title: Implement a program that demonstrate package creation and use in program.

Objective: To demonstrate how to create a package, organize classes into packages, and use them in a Java program.

Steps:

1. **Create a directory structure for the package**:
    - Create a directory with the desired package name, e.g., com/example/utility/.
2. **Create a class inside the package**:
    - Inside the package directory, create a Java class with a package declaration at the top.
    - Example: com/example/utility/Greeting.java.
3. **Write code inside the class**:
    - Add methods and functionality to the class.
    - Example: A sayHello method inside Greeting class.
4. **Compile the class inside the package**:
    - Use javac with the -d option to specify the destination for compiled classes.
    - Command: javac -d . com/example/utility/Greeting.java.
5. **Create another class in a different package or default package (Main program)**:
    - Write a class that will use the class from the created package.
    - Use the import statement to access the class from the package.
6. **Compile the main class**:
    - Compile the class that uses the imported class.
    - Command: javac Main.java.
7. **Run the main program**:
    - Use the java command to run the main class.
    - Command: java Main.
8. **Verify the output**:
    - Check the console output to see if the package and class were used correctly.

Implement a program that demonstrate package creation and use in program.

```
// File: mypack/Message.java
package mypack;

public class Message {
    public void show() {
        System.out.println("Hello! This message is from the mypack package.");
    }
}


// File: PackageDemo.java
import mypack.Message;

public class PackageDemo {
    public static void main(String[] args) {
        Message msg = new Message();
        msg.show();
    }
}
```

OUTPUT:-